

SQL Injection Attack

Cybersecurity Deep Dive

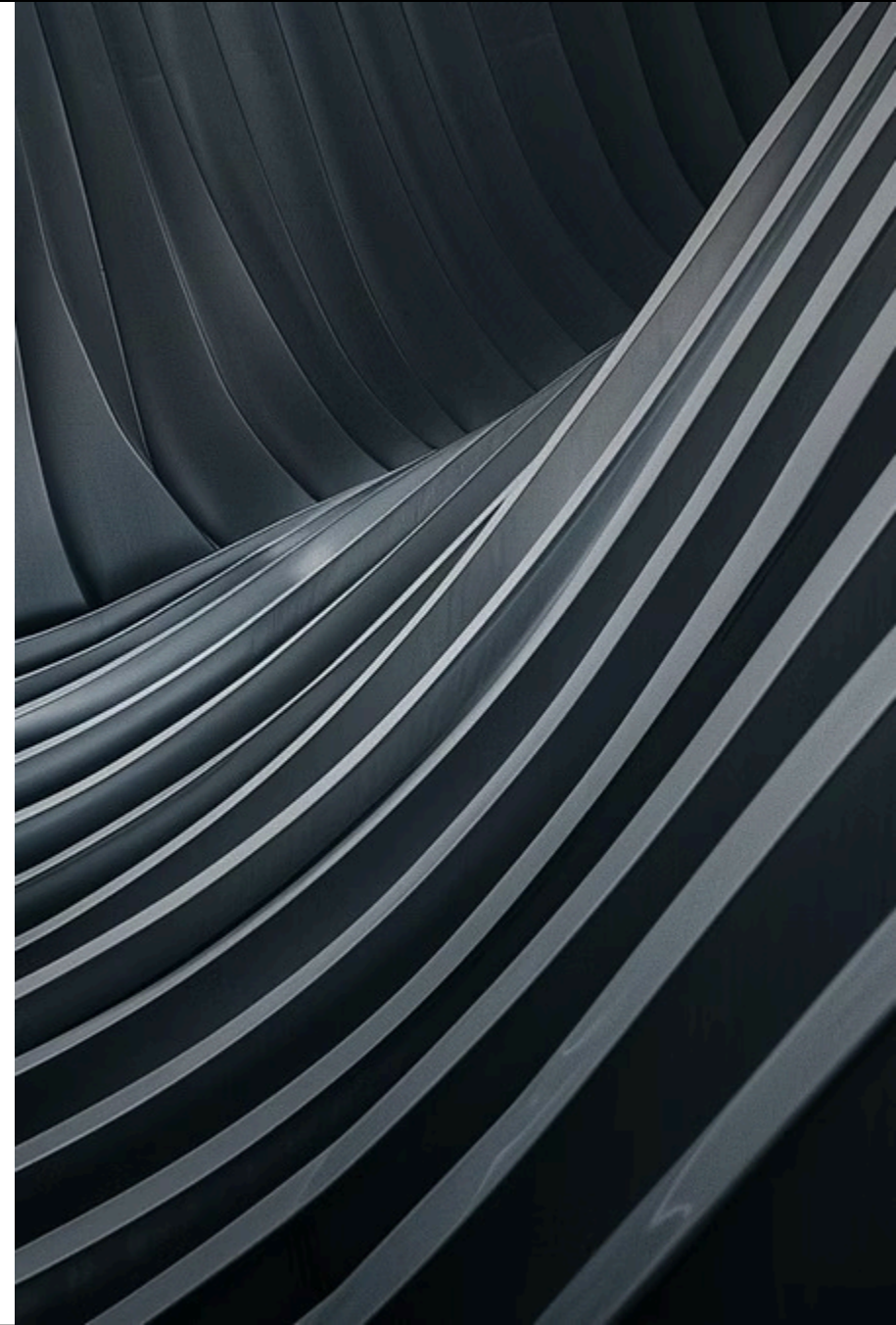
What is SQL Injection?

The Core Concept

SQL injection occurs when an attacker inserts **malicious SQL code** into an application's input field (e.g., login form, search box) to manipulate the backend database. Instead of treating input as *data*, the system mistakenly executes it as *code*.

How It Works

- 1 — Attacker enters malicious input into a vulnerable field
- 2 — Unsanitized input is concatenated directly into an SQL query
- 3 — Database executes the injected code, bypassing intended logic



Why is SQL Injection Used?



Bypass Authentication

Attackers log in as admin without valid credentials by forcing SQL conditions that always evaluate to TRUE, e.g. `' OR '1'='1`.



Extract Sensitive Data

PII, credentials, and financial records are exfiltrated by manipulating SELECT queries to return unauthorized data sets.



Modify or Delete Data

Injected UPDATE and DELETE statements corrupt, alter, or permanently destroy database records, causing operational damage.



Escalate Privileges & Disrupt Systems

Administrative operations can be executed, privileges escalated, and systems disrupted via denial-of-service queries — all from a single input field.

DEPLOYMENT

How is SQL Injection Deployed?

Common Entry Points

SQL injection is deployed through any user input channel that interacts with a database.

Login forms · Search bars · URL parameters (`id=1`) · API requests · Cookies & headers

Classic & Union-Based SQLi

Classic SQLi directly manipulates query logic. **Union-based SQLi** appends a `UNION SELECT` to combine the legitimate result set with stolen data.

```
UNION SELECT credit_card FROM users
```

Blind & Time-Based SQLi

Blind SQLi produces no visible output — the attacker infers data via TRUE/FALSE responses. **Time-based SQLi** uses delays such as

IMPACT

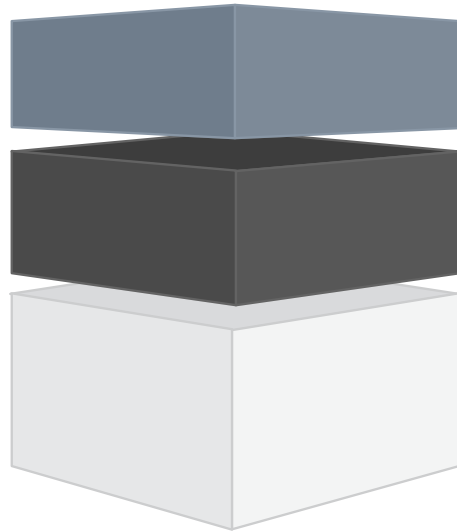
Organizational Impact

Financial Impact

Data breaches trigger GDPR/HIPAA fines, revenue loss, and customer churn.

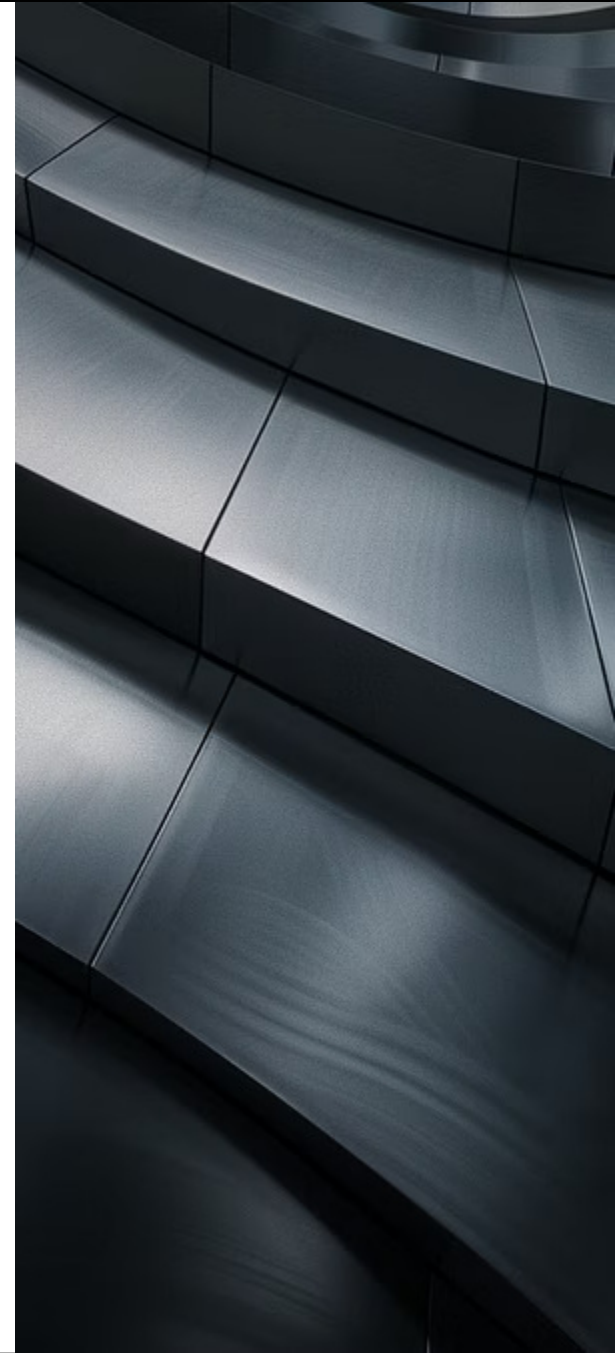
Strategic Impact

Loss of intellectual property, competitive disadvantage, and reputational damage.
Real-world example: Sony Pictures.



Operational & Legal Impact

Database corruption, system downtime, lawsuits, and regulatory penalties.



PREVENTION

How to Prevent SQL Injection



Prepared Statements (Parameterized Queries)

Separate data from code using placeholders: `SELECT * FROM users WHERE username = ? AND password = ?`. This is the single most effective defense.



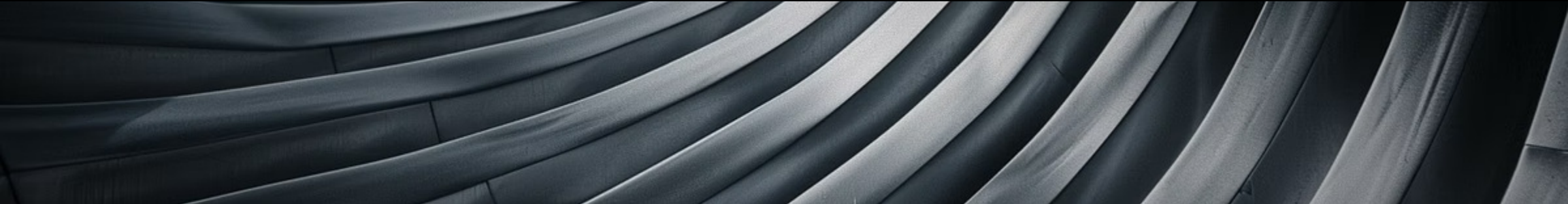
Input Validation & ORM Frameworks

Whitelist allowed inputs and reject suspicious patterns (`'`, `--`, `;`). Use ORM tools like Hibernate or Django ORM that automatically handle query safety.



Least Privilege, WAF & Input Escaping

Grant database users minimal access — never use root accounts. Deploy a Web Application Firewall (WAF) to detect malicious queries, and escape special SQL characters as a last resort.



Mathematical & CS Foundations



Boolean Logic

SQL conditions rely on propositional logic. Attackers exploit OR $1=1$ (always TRUE) and AND $1=0$ (always FALSE) — pure logical manipulation.



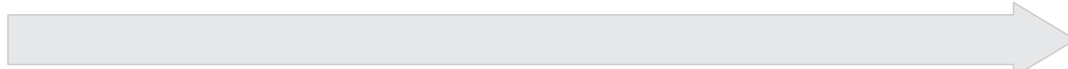
Formal Languages & Parsing

SQL is a formal grammar. Injection breaks the boundary between input (string/data) and query (grammar/code) — a classic **lexical ambiguity** exploit from Theory of Computation.



Probability & Information Theory

Blind SQLi uses timing responses as hypothesis tests — inferring TRUE/FALSE conditions to maximize information gain with minimal queries, reducing entropy about hidden data.



Set Theory & Automata

SQL operations map directly to set theory (SELECT = subset, UNION = union, JOIN = mapping). Injection also exploits state machine transitions — forcing a web app from unauthenticated → authenticated state.

Key Takeaway & Questions

SQL Injection is fundamentally a **failure to separate data from executable logic**. Enforce strict input control, proper query design, and least privilege — and you eliminate ~90% of SQL injection risks.

The Engineering Perspective

Think of SQL injection as a data integrity failure, a process control failure (DMAIC → poor Control phase), and a boundary violation between system inputs and the execution layer.

The Three Pillars of Defense

Strict input control

Proper parameterized query design

Least privilege database access

